AQPIM: Breaking the PIM Capacity Wall for LLMs with In-Memory Activation Quantization

Kosuke Matsushima†, Yasuyuki Okoshi†, Masato Motomura†, and Daichi Fujiki†
Institute of Science Tokyo†
{matsushima.kosuke, okoshi.yasuyuki, motomura, dfujiki}@artic.iir.isct.ac.jp

2026 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-9302-5/26/$31.00 ©2026 IEEE | DOI: 10.1109/HPCA68181.2026.11408452

***Abstract*—Processing-in-Memory (PIM) architectures offer a promising solution to the memory bottlenecks in data-intensive machine learning, yet often overlook the growing challenge of activation memory footprint. Conventional PIM approaches struggle with massive KV cache sizes generated in long-context scenarios by Transformer-based models, frequently exceeding PIM's limited memory capacity, while techniques like sparse attention can conflict with PIM's need for data locality. Existing PIM approaches and quantization methods are often insufficient or poorly suited for leveraging the unique characteristics of activations. This work identifies an opportunity for PIM-specialized activation quantization to enhance bandwidth and compute efficiency.**

We explore clustering-based vector quantization approaches, which align well with activation characteristics and PIM's internal bandwidth capabilities. Building on this, we introduce AQPIM, a novel PIM-aware activation quantization framework based on Product Quantization (PQ), optimizing it for modern Large Language Models (LLMs). By performing quantization directly within memory, AQPIM leverages PIM's high internal bandwidth and enables direct computation on compressed data, significantly reducing both memory footprint and computational overhead for attention computation. AQPIM addresses PQ's accuracy challenges by introducing several algorithmic optimizations. Evaluations demonstrate that AQPIM achieves significant performance improvements, drastically reducing of GPU-CPU communication that can account for 90∼98.5% of decoding latency, together with 3.4× speedup over a SOTA PIM approach.

I. Introduction

Large-scale machine learning models, such as Large Language Models (LLMs), have demonstrated remarkable capabilities [19], [21], [22], [32], [43], [61]. However, their escalating data demands are driving up the cost of data movement, which challenges operational efficiency and sustainability [29]. While conventional optimizations have focused on static parameters such as weights [3], [6], [8], [46], [54], [60], [65], [70], the primary challenge has shifted to the dynamically generated activation (KV) cache, whose memory footprint grows with the volume of contextual data during inference. This trend is further accelerated by the demand for complex reasoning tasks that require longer contexts [21]. Unlike pre-defined weights, these dynamic activations demand real-time optimizations and close collaboration between hardware and software to be managed efficiently [59], [76].

The emergence of Processing-in-Memory (PIM) has opened up significant avenues for mitigating the data movement problem [12]–[15], [24], [35], [39], [56]. By executing computations closer to or within memory, PIM enables a spectrum of optimizations, from accelerating localized arithmetic kernels near banks [24], [35], [39], [56] to performing complex matrix operations directly in memory [7], [12], [14], [57]. Recently, PIM architectures have been increasingly applied to optimize memory-bound attention mechanisms in Transformers and LLMs [24], [56]. A critical application is the autoregressive decoding phase of LLMs, where the GEMV operation between a new token and the KV cache is memory-bound and performing it in-situ effectively alleviates the performance bottleneck [24], [56].

Despite the potential of PIM, several challenges and missed opportunities remain unaddressed. First, current PIM approaches primarily focus on computational gains by leveraging internal memory bandwidth for attention and matrix operations [24], [56], overlooking the critical challenge of memory capacity. Existing PIM architectures struggle with the massive KV cache sizes generated, especially in long-context scenarios (potentially hundreds of GBs) [2], [4], [59]. This frequently exceeds PIM's memory capacity, which is already reduced due to the huge density costs of implementing bank-level PIM [39], creating a fundamental *capacity wall*. Consequently, even SOTA accelerators require a prohibitively large number of devices to accommodate the entire context in PIM (e.g., 40 HBMs for short contexts [56]), exposing a critical scaling issue that current PIM designs overlook.

Conventional methods to mitigate this memory pressure are unfortunately ill-suited for the PIM paradigm. While techniques such as offloading or sparse attention can reduce memory footprint on conventional systems [44], [45], [59], [63], [72], [76], these techniques are not effective for PIM due to their scattered access patterns [67], directly contradicting the need for data locality and contiguous memory access required for efficient computation in PIM [35], [39], [56].

This naturally leads to considering on-chip compression via quantization. However, this path reveals a critical roadblock for PIM. Implementing mainstream quantization schemes [11], [23], [26], [41], [42], [47], [58], [64], [75] directly in PIM requires additional hardware to existing FP16 MAC units for numerical scaling and controls, which incurs a prohibitive area overhead (∼126% just for FP16+INT32 [38]), destroying the memory density. Thus, simply adding quantization logic to PIM would not be a viable solution. Moreover, existing non-PIM hardware-based quantization methods [17], [77] have primarily focused on value-by-value quantization, assuming the orthogonality of weights, but have failed to exploit the inherent

locality and similarity in the context-dependent distributions of activations, missing significant compression opportunities. Therefore, a significant research gap exists in optimizing KV quantization specifically for the distinct attributes of PIM architectures, i.e., high internal bandwidth, low off-chip bandwidth, and limited computational resources and area.

This paper presents AQPIM, a novel PIM-aware activation compression technique that resolves these challenges through a synergistic algorithm-hardware co-design. Our approach is built upon Product Quantization (PQ) [34], a clustering-based method that effectively captures the inherent similarity and locality within activations. While clustering-based quantization has long been recognized for its efficiency, its significant bandwidth requirements have limited its applicability, making it impractical for online use in conventional architectures [67]. The core insight of our work is to turn this long-standing challenge into a key opportunity: we repurpose PIM's massive and often underutilized internal bandwidth to service the intense demands of online clustering. This strategic move makes a superior but previously inaccessible algorithm practical, breaking the capacity wall without costly hardware modifications.

AQPIM's efficiency is realized through several key innovations. First, it transforms the expensive GEMV operation in attention into a sequence of efficient lookups and summations that operate directly on the compressed data, eliminating the need for dequantization and allowing the use of existing simple FP16 MAC units. The primary bottleneck of this lookup-based method—the random access penalty for retrieving centroid data—is solved by a HW co-design. We algorithmically ensure lookup tables reside within a single DRAM row and architecturally add minimal hardware for fast indirect addressing, turning random logical accesses into predictable row-buffer hits. Combined with further algorithmic improvements such as importance-aware clustering, AQPIM achieves high accuracy and performance.

The key contributions of this work are as follows:

- A novel co-design that enables practical, high-fidelity online clustering-based quantization by leveraging PIM's massive internal bandwidth.
- A PIM-aware attention mechanism that computes directly on compressed data, resolving the critical random-access bottleneck through a tight co-design of page-aware clustering and minimal hardware support.
- Algorithmic enhancements and optimizations to maximize clustering accuracy under high compression ratios with minimal overhead.
- The fully integrated AQPIM framework, demonstrating drastic reduction of GPU-CPU communication that can account for 90∼98.5% of decoding latency, together with 3.4× speedup over a SOTA PIM approach due to aggressively reduced KV cache size.

II. Background and Motivation

A. PIM Acceleration for LLM Inference

The escalating demand for processing longer texts with LLMs has led to a significant expansion of their context window sizes. Modern models like Llama 3 [19], GPT-4o [52] and Gemini 1.5 [61] support context windows of up to 128K, 128K, and 1M tokens, respectively. This trend substantially increases the memory access demands. Furthermore, recent advancements in LLMs enable them to generate longer outputs for complex tasks such as reasoning [5], [16], [18], [62], where the model produces detailed explanatory tokens to clarify the logical process, leading to longer decoding output.

Processing long-context in LLM inference presents a critical bottleneck, particularly within the attention mechanism. Attention layers are frequently memory-bound, especially during the decoding phase. This is because each new token generation requires accessing the KV cache, which stores all previously generated tokens. As the context window grows, the KV cache size scales linearly, demanding significant memory bandwidth to fetch these large data structures for GEMV operations.

Recognizing this, prior PIM accelerators such as AttAcc! [56] and NeuPIMs [24] specifically target attention computation as the primary performance bottleneck for LLM inference, and apply PIM for memory-bound operations. AttAcc! proposes a heterogeneous system that combines the powerful computational capabilities of GPU with the high intra-memory bandwidth of PIM. NeuPIMs, on the other hand, integrates NPU with PIM, aiming to maximize concurrent processing by employing a dual row buffer architecture for simultaneous data access and a sub-batch interleaving for fine-grained pipelining.

However, existing PIM architectures struggle with a critical challenge: the sheer volume of KV caches generated, particularly in long-context scenarios, which can reach hundreds of GBs. This often surpasses PIM's memory capacity, especially since *implementing bank-level PIM incurs costs that reduce memory density*. Without mitigation, this can lead to out-of-memory (OOM) crashes, making KV cache compression or offloading crucial for ensuring efficient and scalable performance.

B. KV Cache Mitigation

Several approaches have been proposed to reduce the memory footprint of the KV cache in the device memory: *eviction*, *offloading*, and *quantization*.

Eviction: Following both static [3], [8], [65], [70] and dynamic eviction strategies [1], [33], [40], [53], [74], eviction-based methods keep important tokens to enable lightweight attention computations. StreamingLLM [65] introduces a static token eviction rule that retains certain initial tokens, referred to as *sink tokens*, along with a sliding window, since these tokens tend to produce high attention scores. SnapKV [40] dynamically calculates token importance scores by aggregating recent attention scores during prefilling, and then selects the top-k tokens based on these scores, where k is the number of tokens to be retained. As a result, KV cache becomes sparse, reducing both memory usage and computational cost in the attention mechanism. However, eviction-based approaches inherently suffer from irreversible token loss, leading to accuracy degradation, particularly in long-context scenarios.

Offloading and Sparsification: To mitigate this irreversible token loss, offloading methods [44], [45], [59], [63], [72], [76] preserve the entire KV cache within the memory hierarchy, including the host's main memory and storage. To minimize the overhead of accessing low-bandwidth memory, offloading methods often employ *sparse attention*, which selects only the important tokens for attention computation, thereby reducing data transfer from low-bandwidth memory. Various approaches to vector similarity search, including PQ [72] and graph-based ANNS [44], are employed to identify important tokens, and by retrieving only a small subset of KVs from the host memory, they minimize data transfer overhead. However, these approaches suffer from the bandwidth limitation of the external memory and scattered memory access patterns. For example, NVIDIA's H100 GPU [51] has approximately 26× lower bandwidth for CPU memory access compared to that of HBM. In addition, sparse attention methods often struggle to maintain memory efficiency in Grouped-Query Attention (GQA) and Multiple-Query Attention (MQA) because the shared KV-cache requires accessing the union of KV selections from all query heads within a group, requiring high memory access [67].

Quantization: Quantization [17], [77] has been widely explored for model compression in neural networks, and has also been applied to KV cache compression [11], [23], [26], [41], [42], [47], [58], [64], [75]. It can be categorized into uniform and non-uniform quantization depending on how the original value is mapped into a quantized value.

Uniform quantization maps a real value into an integer by rounding to the nearest integer within fixed intervals. A group of integer values is multiplied by a scaling factor to align the distribution of quantized values with the original. Despite its small bandwidth requirements, it suffers from accuracy degradation, especially when applied to KV cache due to its outliers. Existing works mitigate this problem by increasing the granularity of quantization [23], [42], optimizing granularity depending on the target [47], or smoothing outliers [41], [58], [64]). In addition, achieving inference acceleration often necessitates the quantization of model weights as well, which may result in accuracy degradation.

Non-uniform quantization [26], [27], [68], [69] maps the original distribution into non-uniform datatypes. KVQuant [26] determines the datatypes by using calibration data. It minimizes the quantization error in calibration datasets to obtain optimal quantization values. M-ANT [27] introduces an adaptive numerical type that can support diverse distributions. This also leverages calibration datasets to determine the distribution of KV cache to avoid computational overhead. However, the requirement of calibration datasets potentially leads to sub-optimal performance, especially when processing inputs with different distributions.

The cluster-based approach can mitigate this problem by adjusting quantization values to match the original distribution on the fly. Rather than relying on calibration data, it directly computes quantization centroids from the input data, resulting in accuracy-optimal methods for a specified bit-width. Despite its compression efficiency, it has been considered to be impractical due to the required bandwidth coming from the iterative process for centroid calculation [67]. Thus, its application is typically limited to weight-only quantization with a limited granularity of quantization [68], [69].

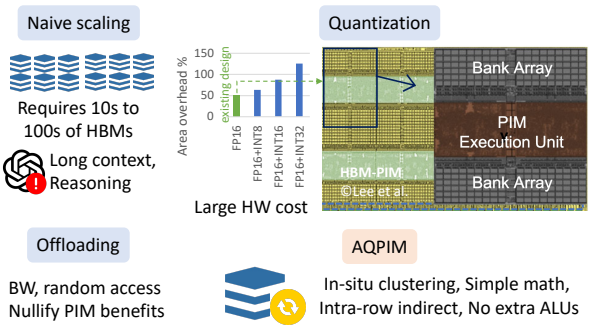


Fig. 1: Scaling challenges of existing PIM designs for LLMs. The die photo is taken from the HBM-PIM paper [39].

C. Motivation

1) The PIM Capacity Wall and Quantization Dilemma: While PIM architectures show promise for accelerating LLM inference, they face a fundamental *capacity scaling problem*, as illustrated in Figure 1. Current PIM accelerators are primarily performance-focused, often assuming the entire KV cache fits within PIM's limited on-chip memory. This assumption quickly breaks down in long-context scenarios, where the KV cache can swell to hundreds of GBs. For instance, a SOTA accelerator like AttAcc! [56] already requires as many as 40 HBM-PIM devices to support even short contexts. This number becomes prohibitively large and economically unviable for the long-context scenarios targeted by modern LLMs.

A seemingly obvious solution—offloading the excess KV cache to the host memory hierarchy—is not a feasible remedy. The high communication overhead of traversing the PCIe bus would nullify PIM's performance gains. Furthermore, sparse attention techniques often used in conjunction with offloading introduce scattered, irregular memory access patterns that directly conflict with PIM's architectural need for data locality to achieve high utilization of its PEs.

This leads to the consideration of data compression techniques, such as quantization. However, implementing conventional quantization methods directly within PIM presents a critical dilemma. Mainstream quantization/dequantization schemes often rely on integer or mixed precision arithmetic (e.g., INT16/INT32 with FP16 [42]) and complex scaling operations. Integrating the necessary compute units for these operations into the already area-constrained bank-level PEs would incur a prohibitive area overhead. As noted in prior work [35], [39], simply adding these integer MAC units could increase the logic area from 50% (FP16 only) to as much as 126% (FP16+INT32), severely compromising memory density. Thus, simply adding new hardware to PIM is not a practical path forward.

2) Our Goals: The challenges identified above define a clear set of principles that must guide the design of a truly practical and scalable PIM-based LLM accelerator. This work introduces AQPIM, a framework built upon the following core design goals:

High-Ratio Compression with High Fidelity in PIM. The primary goal is to break the capacity wall. This requires a compression technique that can drastically reduce the KV cache footprint while preserving model accuracy. Our key insight comes from observing the fundamental properties of activations. The distribution of activations is highly context-dependent, exhibiting significant *locality and similarity*. This is visually demonstrated in Figure 2 using UMAP [49], a dimensionality reduction technique that preserves data's topological structure. As illustrated, key and value vectors exhibit a non-uniform distribution with tight clusters, unlike the more evenly distributed weight vectors. This inherent locality makes the KV cache particularly well-suited for clustering-based quantization such as Product Quantization (PQ), as clusters can naturally adapt to the underlying data distribution.

Synergistic Algorithm-Hardware Co-design. The solution must actively leverage PIM's unique strengths, not just work around its constraints. While powerful, clustering-based quantization has been considered impractical for on-the-fly use due to its massive bandwidth demand in conventional systems. Our design turns this challenge into an opportunity by repurposing PIM's large, underutilized internal bandwidth to service the demands of online clustering. This synergy makes a superior but previously inaccessible algorithm practical, achieving both high compression and high accuracy.

Efficient Computation with Minimal Area Overhead. With online clustering made feasible, the next challenge is performing attention computation without introducing significant logic to the PIM PEs. We introduce a technique to transform the expensive GEMV operations into a sequence of localized lookups and summations of centroids, which is inherently suited for the simple FP16 MAC units already present in PIM. A key advantage of our approach is that it operates directly on the compressed data, eliminating the need for a separate dequantization. The two potential bottlenecks of this approach, i.e., random lookup latency and data growth, are solved via a tight algorithmic and architectural co-design. To eliminate lookup latency, we employ a *page-aware windowed clustering algorithm*. This method maps tokens within a sliding context window to a compact set of centroids that are guaranteed to reside within a single DRAM row. Architecturally, we then introduce a minimal hardware enhancement (indirect addressing in the row buffer) to capitalize on this data locality, making every lookup a fast row-buffer hit. This, combined with an efficient page-aware strategy to update token indices as the context grows, makes the entire computation efficient on existing hardware.

AQPIM is the realization of these design goals, offering a comprehensive framework that enables efficient and flexible activation quantization for next-generation LLM inference on PIM architectures.

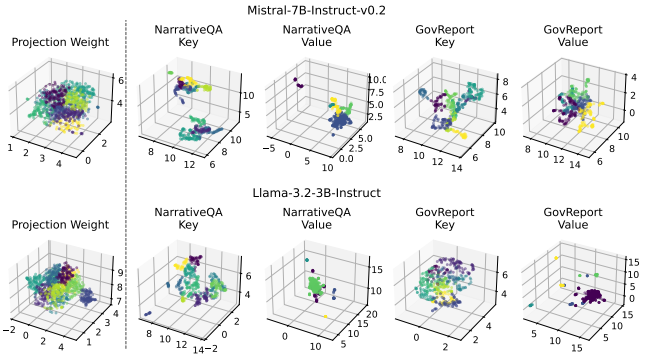


Fig. 2: Locality within the projection weights (left-most) and KV cache (right-four) visualized by UMAP [49], using Mistral-7B-Instruct-v0.2 [32] and Llama-3.2-3B-Instruct [19] running NarrativeQA [37] and GovReport [28]. We present the full results in [71].

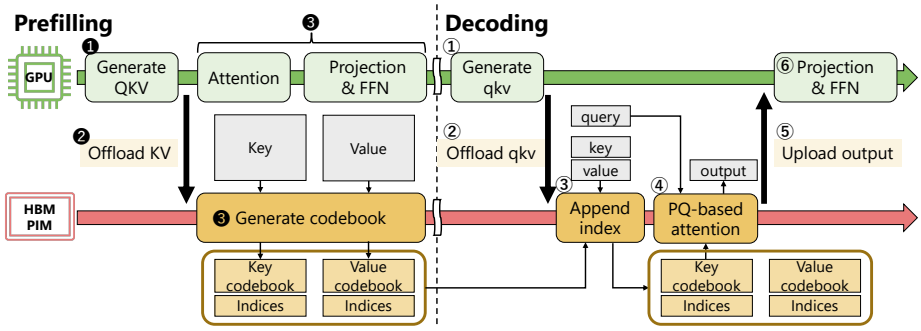
III. AQPIM

A. Overview

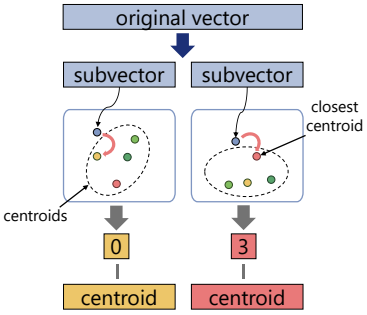
This paper proposes AQPIM, an activation quantization framework utilizing PIM for efficient activation handling and attention computation in large-scale models. Leveraging PIM's high internal bandwidth, AQPIM employs an online, context-aware clustering-based quantization to compress activations. Furthermore, AQPIM uses the resulting *codebooks*, i.e., data structures originally used to reproduce vectors, directly for GEMV. This enables attention computation directly on the compressed data (without decompression) and repetitive reuse of the partial results, significantly reducing both memory footprint and computational overhead within the PIM architecture.

Figure 3a provides an overview of AQPIM. Similar to prior work [24], [56], AQPIM leverages both the high computational power of GPUs and the high intra-memory bandwidth of PIMs. During the prefilling, GPU generates the QKV matrices ❶ and offloads KV to PIM ❷. Then, GPU computes attention and processes projection and feedforward network (FFN) ❸. Meanwhile, PIM generates the *codebooks* and mapping indices with key and value clustering and compression ❸ in parallel with the GPU execution. During the decoding, GPU generates the qkv vectors (hereafter, vectors are expressed with lower case) ① and sends them to PIM ②. Subsequently, PIM appends their indices ③ and computes the attention output using the compressed format ④. Finally, attention output is transferred back to the GPU ⑤, followed by GPU's processing projection and feedforward operations ⑥.

The sequential GPU-PIM processing during the decoding phase may cause GPU idling, especially when the context gets long. This is mitigated by sequence-by-sequence pipelining, where GPU generates query, key, and value vectors for each sequence and immediately offloads them to the PIM, while it proceeds to process the next sequence.



(a) AQPIM execution flow during prefilling and decoding with GPU and HBM-PIM.



(b) PQ applies clustering-based quantization.

Fig. 3: Overview of AQPIM and Product Quantization (PQ).

B. Product Quantization

As motivated in Section II-C, AQPIM is designed to: (a) leverage context-dependent similarity and locality through its quantization scheme, (b) achieve a balanced trade-off between compression efficiency and PIM-suitable bandwidth, (c) effectively utilize the localized memory scope of PIM, and (d) minimize both memory footprint and computation within the PIM architecture. To this end, we adopt Product Quantization (PQ) as our baseline quantization technique.

PQ is a vector quantization technique widely recognized in the approximate nearest neighbor search (ANNS) for its capacity to significantly compress high-dimensional vector data while preserving locality and similarity. As illustrated in Figure 3b, PQ has two fundamental characteristics: (1) *vector splitting* and (2) *clustering-based quantization*. (1) *Vector Splitting* decomposes high-dimensional vectors into smaller subvectors. This allows for parallel processing across subvector groups, effectively utilizing PIM's high parallelism and localized memory scope, while improving expressibility by combining multiple subvector spaces to reconstruct a vector. (2) *Clustering-based quantization* significantly reduces quantization error by exploiting similarity and locality in data distribution. PQ typically employs k-means clustering, which partitions vectors into k clusters based on the Euclidean distance. Since distance calculation for each vector is independent, this approach aligns well with the parallel processing capabilities of PIM.

Clustering has been used to *identify* important tokens in offloading-based sparse attention approaches like PQ-Cache [72] and Squeezed Attention [25], where a *full exact copy of KVs is kept at and fed from CPU*. Importantly, AQPIM *directly* uses PQ as a quantization method and KV source in PIM without the full KV copy. This eliminates the need for CPU offloading and bandwidth for KV transfers, while we observe that the naive adoption of PQ as a KV source results in a non-trivial accuracy drop. This will be addressed by our algorithmic techniques introduced in Sections III-C and III-D.

PQ for KV Cache Quantization: PQ's codebook generation can be a significant bottleneck when applying PQ for KV cache quantization during inference. To overcome this issue, we leverage underutilized PIM resources during the prefilling stage. Figure 3a shows parallel processing of GPU and PIM. PIM generates the codebooks concurrently with GPU computation during the prefilling stage.

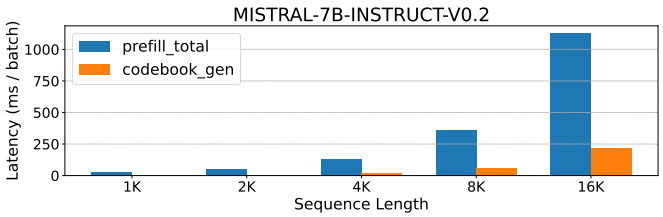


Fig. 4: The latency comparison of the prefilling and the clustering process in 128 head-dimensional space.

To keep up with the GPU's throughput, codebook generation must be completed before the GPU offloads KV of the subsequent layer. While standard k-means iterates cluster reassignment until convergence, our experiments demonstrate that just *four* iterations converge to a stable state and yield comparable accuracy, effectively hiding the clustering process behind the GPU's computations. The clustering overhead can be completely hidden regardless of sequence length, as shown in Figure 4. Given a vector PEs, while the latency for attention scales with N^2, clustering scales with $n_{\text{centroids}}N$, where $n_{\text{centroids}}$ is a constant and $n_{\text{centroids}} \ll N$. Furthermore, peak memory usage is minimized by layer-wise codebook generation, enabling sequential compression of KV cache.

PQ for Efficient Attention Computation: PQ significantly optimizes attention computation by directly leveraging codebook representations. In PQ, the key cache is decomposed into a key codebook and a set of key indices. The key codebook contains centroids generated during the prefilling stage, and the key indices indicate the centroid assignment for each original token. Since our goal is to compute the inner product of the query vector and the key matrix, the reconstruction of the full key matrix can be skipped. Figure 5 illustrates this skip method. The query vector is first divided into m subvectors ❶. Subsequently, each subvector is multiplied with its corresponding codebook's submatrix, collectively forming an inner product matrix ❷. The key indices, indicating centroid

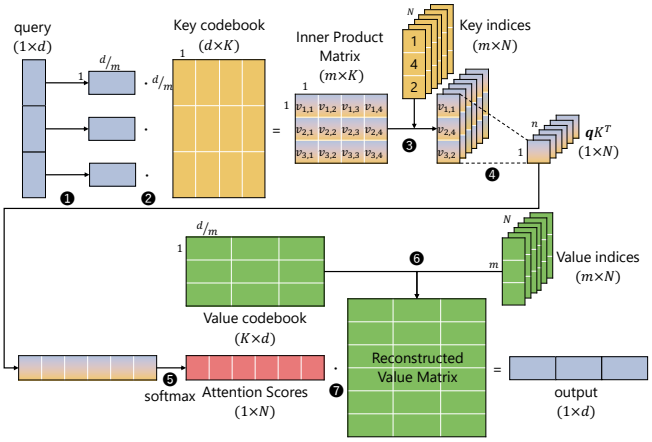


Fig. 5: Computation flow of PQ-based attention. Matrix multiplications are simplified by inner product matrix lookup and summation.

assignments, are then used to lookup values in the inner product matrix ❸. The retrieved values are summed along the vector splitting axis ❹, which produces an approximation of the inner product $\boldsymbol{q}K^T$. This sequence of operations ❶–❹ dramatically reduces the computational cost of obtaining $\boldsymbol{q}K^T$ by avoiding the explicit multiplication between the query and the full key matrix. Subsequently, the softmax function is applied to $\boldsymbol{q}K^T$ to produce the attention scores ❺. The value matrix is reconstructed using the value codebook and value indices ❻. Finally, the output vector is computed as the inner product of the attention scores and the reconstructed value matrix ❼. This method achieves substantial computational savings by replacing large-scale matrix multiplications with efficient, localized codebook lookups and summations. Our PQ-based attention mechanism is orthogonal to recent techniques such as Grouped-Query Attention (GQA) and Multiple-Query Attention (MQA).

Mitigating Random Access for Efficient Lookup: A naive implementation of the inner product matrix lookup in PQ-based attention generates irregular memory accesses that lead to frequent DRAM row activations.

To address this issue, we propose a *page-aware windowed clustering* method, co-designed with intra-row indirection support in AQPIM introduced later in section III-F. The core idea is to restrict the indirect access to happen within a single DRAM row in a bank by co-locating all related inner product values.

For example, HBM-PIM architecture has 1KB row buffers in each bank, which stores 512 inner product values (in FP16 format), and we use that many centroids for a given context window so that indirection only happens within a page. Although a single window that maps the entire sequence to 512 centroids suffices for most long-context scenarios, it can be extended for more centroids. To do this, as shown in Figure 6 (1) a sequence is divided into multiple windows, and as the window advances, the previous centroids are copied to a new page and subsequently updated for the window. Then, (2)

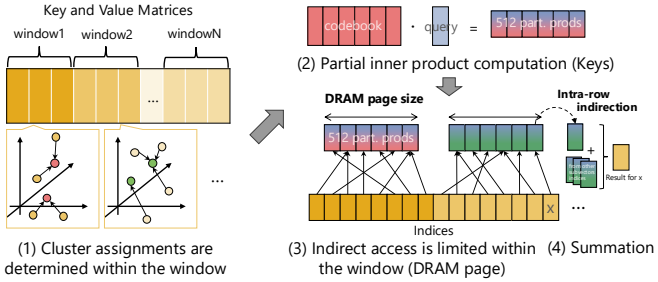


Fig. 6: Page-aware windowed clustering.

partial inner product is computed, and (3) intra-row indirection performs lookups within a DRAM page. Since indirect access within each window is fully contained within a single row, it greatly minimizes the number of row activations, reducing it to as few as the number of windows. The partial results are then (4) summed up with partial products from different subvectors.

This approach is also extended for the value codebook, while we may loop over the value indices multiple times to accommodate the larger tensor dimensions.

C. Weighted Codebook Generation

Despite PQ offering potential benefits of acceleration of attention computation, just applying standard PQ to attention layers leads to substantial accuracy loss. Consequently, prior work PQCache [72] limits its use to identifying important tokens and retrieves their original KVs from CPU memory. We hypothesize that this accuracy degradation stems from PQ's inherent inability to account for the varying levels of importance among different tokens. Previous studies [65], [74] have shown that certain tokens consistently receive high attention scores. These critical tokens play a crucial role in preserving model accuracy, but the conventional PQ treats all tokens equally during quantization.

To address this problem, we propose importance-weighted k-means clustering, ensuring that tokens with higher attention scores are quantized with fewer quantization errors. We modify the k-means clustering process by incorporating attention score-based weighting, giving higher priority to tokens with greater impact on the model accuracy. First, we compute a weight vector $\boldsymbol{w} \in \mathbb{R}^N$ from the attention score matrix $\mathbf{S} \in \mathbb{R}^{N \times N}$. Each weight is the sum of attention scores received from the last t tokens in the sequence:

$$\boldsymbol{w} = \text{sum}(\mathbf{S}[-t:, :], \text{axis} = 0), \tag{1}$$

where t is a tunable window size. The algorithm then iteratively minimizes a weighted objective function, which is the total weighted squared Euclidean distance between each token $\boldsymbol{x}_n$ and its assigned cluster centroid $\boldsymbol{\mu}_k$. In each iteration, tokens are assigned to their closest centroid, and centroids are subsequently updated using a weighted average of their members:

$$\boldsymbol{\mu}_k = \frac{\sum_{n \in C_k} w_n \boldsymbol{x}_n}{\sum_{n \in C_k} w_n}, \tag{2}$$

where C_k is the set of indices for the tokens assigned to the k-th cluster. By introducing these weights, the centroids are more influenced by tokens exhibiting high attention scores, thereby greatly reducing quantization error for these critical tokens. As a result, this approach improves accuracy retention while preserving the benefits of PQ compression.

The weights $\boldsymbol{w}$ defined in Eq. (1) are computed on the GPU during the prefilling phase. Since the attention score $\mathbf{S}$ is used in both attention and weights computations, the additional computational overhead is minimal and aligns with FlashAttention [10].

D. Optimization of Vector Splitting

Standard PQ splits vectors without considering inter-channel similarity, which often leads to higher quantization errors. To address this, we introduce a channel sorting preprocessing step. By grouping highly correlated channels together before partitioning, this approach creates more cohesive subvectors, thereby minimizing quantization error while maintaining the original codebook size.

Our sorting method groups channels based on cosine similarity. The process begins by randomly selecting a reference channel. The cosine similarity is then computed between this reference channel and all other channels. Based on the results, the top-k most similar channels are greedily selected to form a group. These steps are repeated m times, where m is the number of subvectors, until every channel has been assigned to one of the groups. As a result, channel vectors within each group exhibit high mutual cosine similarity. Compared to coupling contiguous channels for quantization [73], pre-sorting helps increase intra-group affinity, reducing quantization error.

This channel sorting operation can be seamlessly integrated into the projection matrices, effectively hiding any associated overhead. Following the approach of [11], we introduce static sorting matrices, P_k and P_v for the key and value vectors. The sorting matrices can be absorbed into the projection matrices. Specifically, they can be incorporated as $W'_q = W_q P_k$, $W'_k = W_k P_k$, $W'_v = W_v P_v$, and $W_o = W_o P_v^T$. Moreover, these sorting matrices are generated offline using a calibration dataset, such as Wikitext-2-v1 [50], thereby avoiding additional runtime overhead during inference.

E. System Architecture

We consider a hardware design based on HBM [31] integrated with PIM. HBM provides high memory bandwidth thanks to its 3D-stacked DRAM dies, which are interconnected via through-silicon vias (TSVs). This 3D architecture also enables high energy efficiency since data can be transferred over much shorter distances. HBM has been adopted in recent GPUs such as NVIDIA's H100 [51], making it a suitable candidate for integration with PIM.

Execution Unit: A key challenge in HBM-PIM design is the placement of PE, which significantly impacts throughput and energy efficiency. To address this issue, AttAcc! [56] explores PE placement in terms of peak power, throughput, energy consumption, and area overhead. Building on this analysis,

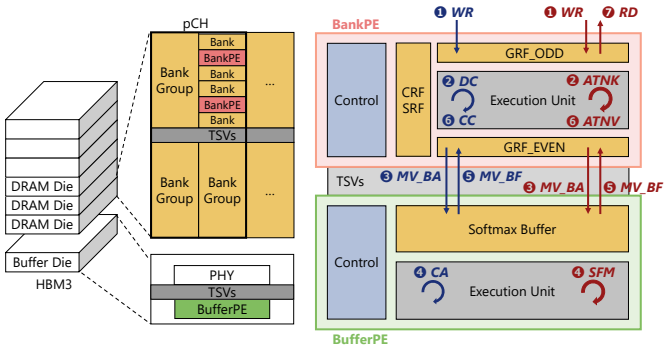


Fig. 7: AQPIM architecture and dataflow.

TABLE I: List of AQPIM operations.

Process	Place	Necessary Unit
Distance Calculation (DC)	BankPE	ADD, MUL, SUM
Cluster Assignment (CA)	BufferPE	MIN
Centroid Calculation (CC)	Both	MUL, SUM, DIV
Attention (ATNK, ATNV)	BankPE	MUL, SUM
Softmax (SFM)	BufferPE	ADD, SUM, MAX, DIV, EXP

we introduce two PE architectures: BankPE and BufferPE, as illustrated in Figure 7. BankPE is placed adjacent to the DRAM banks, leveraging the high internal bandwidth while facing strict area constraints. In contrast, BufferPE is located in the buffer die of HBM. Although it provides lower bandwidth compared to BankPE, it benefits from fewer area constraints. Moreover, BufferPE is particularly advantageous for data-intensive operations that involve inter-bank data movement, which would otherwise introduce bottlenecks in BankPEs. Importantly, we design the microarchitectures for BankPE and BufferPE based on their individual strengths and limitations. While we utilize the architectural modules based on AttAcc!, we optimize them to be well-suited for our AQPIM algorithms.

We identify the necessary computation unit for PQ and attention, as shown in Table I. Distance calculation (DC), centroid calculation (CC), and attention computation (ATNK, ATNV) are not data-intensive. This motivates placing their corresponding computation units in BankPE. Moreover, since some units overlap, frequently used operations can be efficiently assigned to BankPE, minimizing its area overhead. From this perspective, ADD, MUL, and SUM units are placed in BankPE. In contrast, data-intensive operations, such as cluster assignment (CA) and softmax-related operations (SFM), are assigned to BufferPE to reduce inefficient inter-bank data transfers. This placement is also suitable because the DIV and EXP calculators consume relatively large chip area. Note that AQPIM does not introduce any specialized computation units or ALUs for quantization to area constraint BankPEs and only uses existing FP16 MAC units.

Dataflow: Figure 7 also illustrates the data flow for both codebook generation (blue arrows) and attention computation (red arrows). Codebook generation begins with receiving KV matrices from the GPU, which are then distributed to each BankPE ❶. Each BankPE performs distance calculation (DC) ❷, and the results are transmitted to the BufferPE ❸. The

BufferPE then determines cluster assignments (CA) ❹ and returns the assignment results to their respective banks ❺. Based on these assignments, each BankPE performs centroid calculation (CC) ❻. The BankPE computes the numerator (a weighted sum of vectors), while the BufferPE computes the reciprocal of the denominator (a sum of weights), as presented in Eq. (2). The final division is thus reduced to a single multiplication at the BankPE. Steps ❷-❻ are iteratively repeated until the codebook is ultimately generated.

The attention computation begins with the query vector being received ❶. Each BankPE performs multiplication between the query and key codebook (ATNK) ❷, and transfers the results to the BufferPE ❸. The BufferPE then computes the softmax function (SFM) ❹ and returns the results to each BankPE ❺. Finally, each BankPE performs the final attention computation (ATNV) ❻, sending the attention output to the GPU ❼.

Commands: Several dedicated PIM commands are introduced on top of the AttAcc! design to control PQ-related processes. `PIM_SET_CONFIG` broadcasts the PQ configuration, including parameters such as the number of subvectors and the number of centroids. `PIM_MAC_AB` executes MAC operations in all banks. `PIM_SFM` executes softmax-related operations within BufferPE. `PIM_RET` executes row buffer retrieval explained in section III-F. In addition to computational commands, we use several data movement commands. `PIM_MV_BA` command moves data from BankPE to BufferPE, and `PIM_MV_BF` command transfers data from BufferPE back to BankPE. To manage I/O, `PIM_RD` command reads the final results of attention computation from the bank, and `PIM_WR` writes input data to the bank as the initial step of the processing. Finally, to ensure proper DRAM operation, we include system-level commands such as `PIM_ACT_AB` command, which activates DRAM rows in all banks. Although these commands are not yet implemented on HBM-PIM, they are issued through the standard HBM command path in the same manner as conventional DRAM commands.

F. Intra-Row Indirection

Since clustering assigns arbitrary centroids, the following lookup operations during attention computation involve random access to the inner product table. To efficiently manage these irregular memory access patterns, we propose the intra-row indirection architecture, as illustrated in Figure 8. This mechanism operates as follows: First, the row storing the target (inner product) values is activated, and transfer the data into a row buffer. Then, the lookup indices stored in a general-purpose register file (GRF) are redirected to the column decoder. The column decoder then outputs the corresponding values, which are streamed through the existing datapaths to the buffer die or GRF. The BufferPE executes the subsequent softmax operation and transfers the results back to the banks.

Importantly, only a single row activation is necessary as long as the memory address scope represented by indices fits entirely within a row buffer, which is guaranteed by page-aware windowed clustering. Moreover, this mechanism incurs

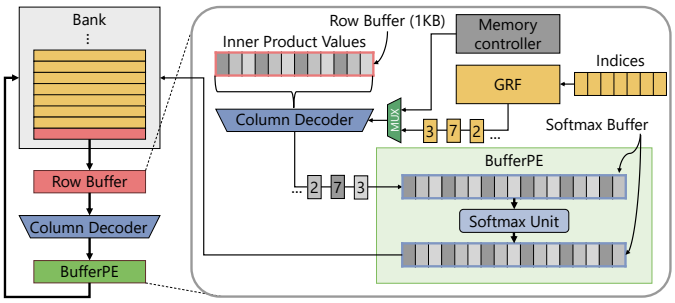


Fig. 8: Intra-row indirection for efficient random access.

no significant additional area overhead, making it well-suited for BankPE, which often faces severe area constraints.

G. Data Mapping Strategy

We design an efficient data mapping strategy to maximize PIM utilization, as illustrated in Figure 9. Attention computation is inherently parallel due to mechanisms such as MHA and GQA. In addtion, PQ's codebook is applied per head. From this, each attention head is mapped to a separate HBM. This head-wise mapping effectively eliminates unnecessary data transfers between HBMs, ensuring a high utilization even when multiple heads share a stack. Moreover, within the AQPIM framework, each head is further split into multiple subvectors. To maximize BankPEs utilization, each subvector should be assigned to a different bank. This implies that PIM utilization is maximized if $N_{\text{subvecs}} > \frac{N_{\text{banks}}}{N_{\text{heads}} \times N_{\text{batches}}}$. It is easy to satisfy this condition with practical LLM setups. This bank-wise data mapping is realized by simply incrementing the address because AQPIM employs an address mapping scheme that utilizes the lower bits of the memory address for bank selection.

H. Memory Allocation

To use memory space effectively and minimize unnecessary memory remapping, the memory region in each bank must be handled properly. We utilize a simple and effective memory allocating strategy. The memory region is allocated before the prefilling phase. The codebook region is fixed and constant, and the buffer region used during prefilling is also fixed and reused for each layer. PQ indices are allocated layer by layer at the page granularity. A fixed-size PQ index region for prefilling phase is allocated between the codebook and buffer regions and overwrites the buffer region thereafter during decoding. These regions need not be dynamically reclaimed, but released at once after completion.

IV. Evaluation

A. Experimental Setup

Models: We conduct experiments using two open-source LLMs: Mistral-7B-Instruct-v0.2 [32] and Llama-3.2-3B-Instruct [19]. These models have long context window sizes of 32K and 128K, respectively. We use bfloat16 for both models, which is a common format for LLM inference.

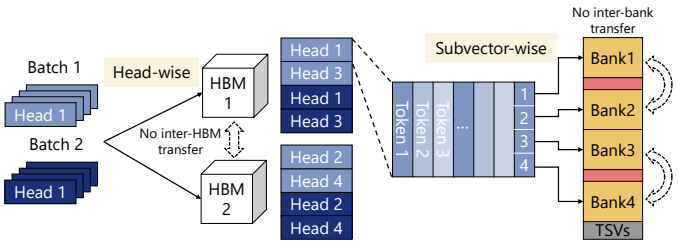


Fig. 9: Data mapping strategy. Each head is assigned to a separate HBM, and each subvector set is mapped to an individual memory bank.

Tasks: We evaluate AQPIM using LongBench [2], a widely used benchmark for long-context LLM inference. LongBench includes diverse task categories such as single- and multi-document question answering, summarization, few-shot learning, synthetic tasks, and code completion. To evaluate overall accuracy trends, we select one representative task from each of these categories for our experiments.

Baselines: We compare AQPIM with SnapKV [40], PQ-Cache [72], and SKVQ [11]. SnapKV is also a state-of-the-art sparse attention method that dynamically selects tokens during inference, enabling efficient long-context processing. PQCache is an offloading method that uses PQ to identify important tokens. It offloads KV cache to CPU memory and retrieves a small number of tokens based on a maximum inner product search with PQ. SKVQ is a state-of-the-art quantization approach that reorders channels to minimize the quantization error within each quantization group.

Hyperparameters: Based on the configurations of PQ-Cache and SKVQ, we retain the first 8 tokens with full precision, which is well known as sink tokens. We adopt the same approach in AQPIM. In addition, similar to other methods, AQPIM preserves the most recent 32 tokens, referred to as sliding window tokens, with full precision. We also use these 32 ($= t$) tokens to calculate weight $\boldsymbol{w}$ defined in Eq. (1). AQPIM also has two key hyperparameters: the number of subvectors and the number of centroids. First, to determine the optimal number of subvectors, we conduct experiments on a subset of LongBench with Mistral-7B-Instruct-v0.2. In these experiments, we vary the number of subvectors m while keeping the number of centroids fixed at 512. As shown in Table II, using 32 subvectors achieves the best balance. Subsequently, we varied the number of centroids, observing that accuracy saturated at 512 centroids, as shown in Table III. This number of centroids (512) is also well-suited for intra-row indirection, explained in section III-F.

Online codebook update: We tried OnlinePQ [66], which progressively updates centroids at each decoding step. However, it is observed to have little impact on accuracy, even on LongBench. Moreover, in some cases, it negatively affected accuracy; for example, the average score of LongBench tasks is 0.36 points lower than that of the non-OnlinePQ configuration. Therefore, we omit OnlinePQ from our experiments.

TABLE II: Accuracy comparison across different number of subvectors m.

Configuration	m=2	m=4	m=8	m=16	m=32	m=64
NarrativeQA	21.45	20.58	21.36	22.09	22.59	21.81
HotpotQA	31.35	32.31	35.71	37.40	37.83	37.85
GovReport	21.05	21.42	22.49	25.91	29.88	30.73
TREC	51.00	57.00	65.50	70.00	71.00	71.00
PRetrieval	86.35	87.13	88.81	88.19	87.69	86.85
LCC	53.83	54.89	55.03	55.45	55.33	55.23
Average	44.17	45.56	48.15	49.84	**50.72**	50.58

TABLE III: Accuracy comparison across different number of centroids K.

Configuration	K=64	K=128	K=256	K=512	K=1024
NarrativeQA	23.25	23.09	21.37	22.59	21.91
HotpotQA	37.38	38.08	38.00	37.83	38.13
GovReport	22.99	25.49	28.53	29.88	30.74
TREC	69.00	70.50	71.00	71.00	71.00
PRetrieval	81.56	88.17	87.85	87.69	87.36
LCC	54.08	54.66	55.17	55.33	55.20
Average	48.04	50.00	50.32	**50.72**	**50.72**

B. Experiments on LongBench

We compare the tradeoff between memory reduction ratio and accuracy of AQPIM against SnapKV, PQCache, and SKVQ. As shown in Figure 10, AQPIM achieves a comparable tradeoff across all tasks and on both models. Compared to SnapKV and SKVQ, AQPIM tends to exhibit a better trade-off, thereby demonstrating that PQ, a clustering-based method, delivers higher quantization quality. While PQCache maintains high accuracy even under aggressive compression ratios by storing the full KV cache in CPU memory and thereby mitigating information loss, AQPIM achieves comparable accuracy up to approximately 80% compression while operating entirely within PIM memory.

C. Ablation Study

To evaluate the effectiveness of the importance-weighted k-means clustering and vector splitting optimization, we conduct ablation studies comparing four configurations: standard PQ, AQPIM without weighting, AQPIM without pre-sorting, and AQPIM. Table IV presents the result for high compression scenarios (128 centroids). We observe that applying both weighting and pre-sorting significantly contributes to accuracy, particularly under aggressive compression situations. The overhead introduced by these techniques is minimal: importance-weighted k-means leverages the attention scores generated during attention computation, and the channel permutation is generated offline using a calibration dataset and integrated into the projection weights during inference.

D. System Simulation

Hardware configuration: We simulate the performance of the AQPIM by comparing a conventional system with GPU+HBMs, GPU+HBM-PIMs (AttAcc! [56]), GPU+AQPIM. All simulated configurations utilize NVIDIA's H100 GPUs [51]. The baseline conventional GPU+HBMs consists of 1 H100 GPU core and 5 16GB HBM modules.

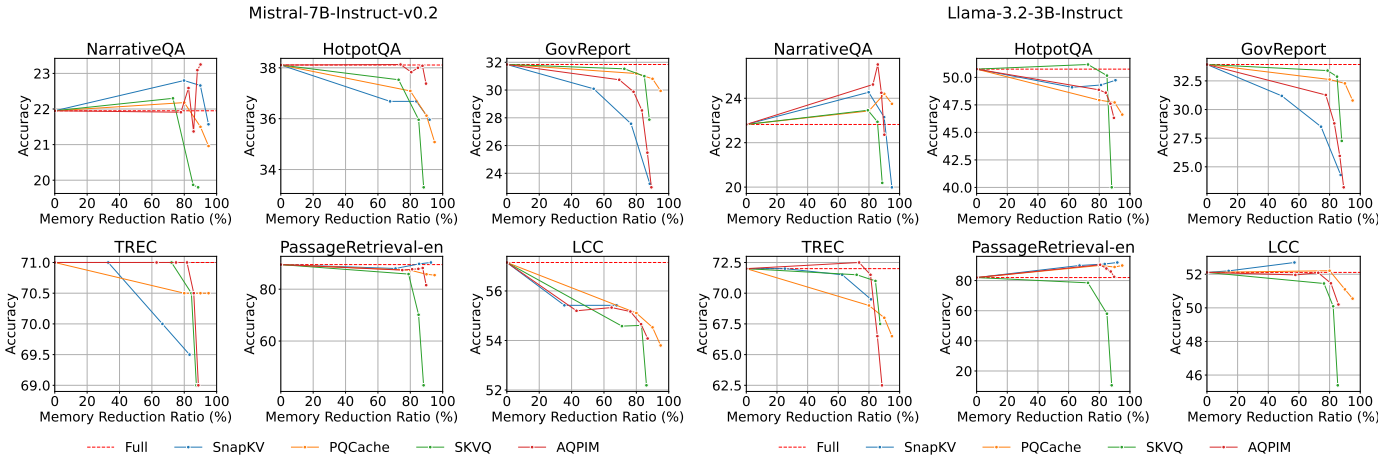


Fig. 10: Memory reduction ratio vs. accuracy.

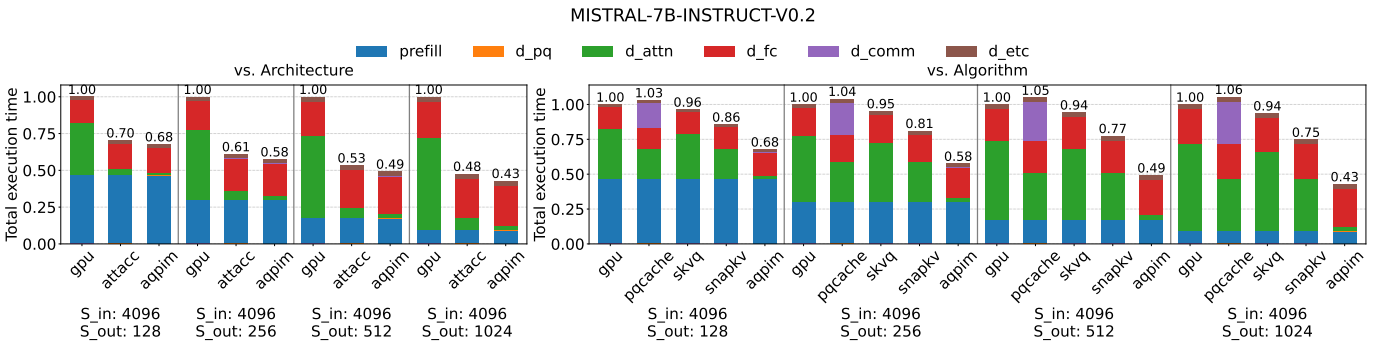


Fig. 11: Normalized total execution time comparing different architectures (left) and algorithms (right).

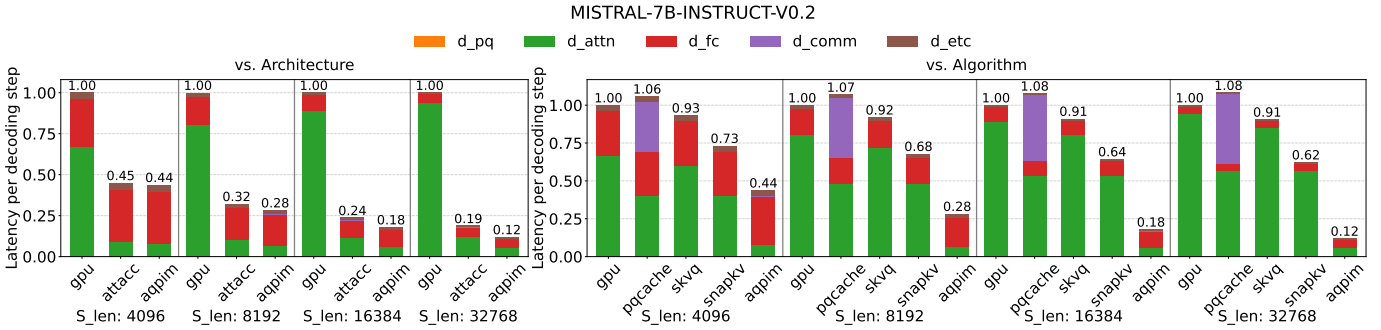


Fig. 12: Normalized decoding time comparing different architectures (left) and algorithms (right).

In the PIM-enabled systems (AttAcc! and AQPIM) replace 4 16GB HBMs with 4 16GB HBM-PIMs. These HBM-PIMs are allocated for KV cache storage, while the remaining HBMs are used for other data, such as model parameters. In addition, we used Intel's Xeon Platinum 8480+ Processor [30] in our experiments with CPU.

Baselines: We compare AQPIM against other architectures and KV cache compression methods. The architectures include GPU+HBMs and AttAcc!, while the KV cache compression methods include PQCache, SKVQ, and SnapKV. For AQPIM, we set the number of subvectors to 32 and the number of centroids to 512. For the other compression methods, we set the memory reduction ratio to 80%.

Note a key difference in assumptions: AttAcc! does not inherently address scenarios where large KV caches overflow the HBM-PIM memory. In contrast, AQPIM operates under the assumption that while the model itself can fit within memory, the full KV cache may not, thus necessitating compression. Importantly, our approach can also scale out to provide a larger memory capacity to accommodate larger models. Even when the dimensionality increases, AQPIM can scale by increasing the number of subvector sets to keep the per-codebook vector space constant. This preserves the expressive power of subvectors and codebooks while enabling scaling with the abundant bank resources.

Simulator: To evaluate these baselines, we constructed

TABLE IV: Effect of introduced PQ optimizations. Both importance-weighted clustering and channel pre-sorting contribute to accuracy improvement particularly under aggressive compression situations.

Configuration	Standard PQ	w/o weighting	w/o pre-sort	AQPIM
NarrativeQA	18.35	20.51	21.42	23.09
HotpotQA	37.06	34.39	36.65	38.08
GovReport	24.45	25.63	23.55	25.49
TREC	70.65	70.15	69.65	70.50
PRetrieval	61.26	55.49	86.47	88.17
LCC	53.94	53.34	54.83	54.66
Average	44.29	43.25	48.76	**50.00**

a customized GPU-PIM simulator for LLM inference, built upon the AttAcc! simulator [55]. This simulator itself is a modified version of Ramulator [20], [36], [48], adapted to simulate LLM inference. The simulator takes three inputs: system configuration, model details, and input configurations, and outputs both execution time and energy consumption.

Energy and Area: AQPIM repurposes most of the existing HBM peripherals (e.g., 1K row buffer and column decoder) and the PE designs implemented in AttAcc! [56]. The added logic for intra-row indirection, which is synthesized with the same PDK [9] and scaled with the same DRAM die density ratio as AttAcc!, is 0.0565mm^2 per HBM. This is only 0.43% of BankPE area of HBM3 implementing AttAcc!. BufferPE in AttAcc! already has all the required components in Table I in their accumulators and softmax units.

Timing and energy consumption used in our simulation are based on the synthesis results reported on AttAcc!. Since the column decoder input is latched for pipelining, t_{CCDL} (delay between RDs to the same BG) will not be affected by our modifications. DRAM traffic and contentions are entirely managed by DRAM controllers and modeled in our simulator.

While our tested model and algorithm employ the de facto standard bfloat16, the architecture simulation is conducted using FP16 to make it directly comparable to prior work [56]. Comparing FP16 and bfloat16 MAC implementations on HBM-PIM, the bfloat16 unit has identical latency but provides 13% smaller area and 14% better energy efficiency per operation [39]. While FP16 can still gain considerable benefits from our approaches, bfloat16 would also be a compelling option for future HBM-PIM designs.

Scenarios & Models: Our experiments evaluate a range of scenarios by varying input lengths, output lengths, and batch sizes. Unless otherwise specified, we use a batch size of 16. The evaluated model is Mistral-7B-Instruct-v0.2 [32].

E. Performance

We first compare the total execution time of the different approaches. Figure 11 presents the normalized total execution time for a 4K input, varying the output length. The left graph compares the performance of GPU, AttAcc!, and AQPIM. AQPIM significantly reduces the matrix multiplication (matmul) execution time during the decoding phase. This reduction ratio increases with longer output lengths, achieving up to a 2.33× faster overall execution time.

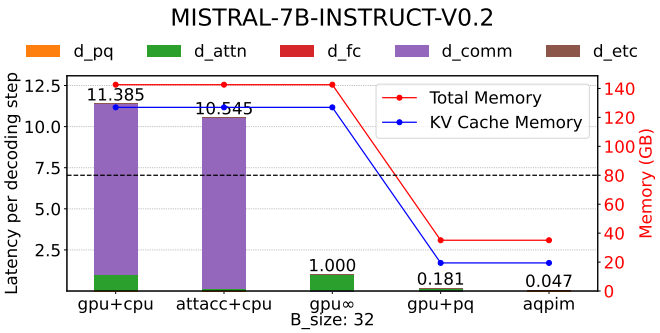


Fig. 13: Decomposition analysis of decoding speedups. `gpu∞` is a baseline that assumes infinite GPU memory capacity. Both `gpu+pq` and `aqpim` utilize PQ compression.

The right graph compares PQCache, SKVQ, SnapKV, and AQPIM. All compression methods, except for AQPIM, were executed on GPU, since their designs are not primarily intended for PIM. As illustrated, PQCache experiences performance limitations due to communication overhead with CPU memory. SKVQ achieves some speedup over the GPU baseline due to the reduced data transfer, but its acceleration is limited since current GPUs do not efficiently handle quantized low bit values, often requiring upcasting to larger bit precision. SnapKV demonstrates better acceleration than other methods, but its performance gain remains considerably smaller than that of AQPIM. This is attributed to AQPIM's ability to leverage both architectural acceleration from PIM and algorithmic optimizations from PQ-based attention.

Moreover, as the output length increases, the decoding phase becomes dominant in the total execution time. Therefore, our subsequent analysis focuses solely on the decoding process.

We secondly compare the execution time of each decoding step, varying the input length. Figure 12 presents the normalized execution time per decoding step. The left graph compares different architectures, and the right graph compares algorithms. AQPIM greatly reduces matmul execution time compared to both architectures and algorithms, achieving up to an 8.33× speedup, and this effect becomes more notable as the input length increases. This is because AQPIM maintains a fixed number of centroids at 512, which fit within a DRAM row, thus ensuring a constant matmul cost. Although the retrieval cost increases with sequence length, this cost remains negligible due to our efficient intra-row indirection mechanism, as discussed in Section III-E.

We further analyze the speedup breakdown of AQPIM. In this analysis, we will also assume an imaginary GPU with infinite memory capacity (`gpu∞`) to separate the contribution from GPU-CPU communication reduction of AQPIM. Figure 13 presents four evaluation scenarios: `gpu+cpu` (GPU offloads KV cache to CPU when it overflows), `gpu∞` (infinite GPU memory, no offloading penalty), `gpu+pq` (PQ compressed KV with GPU), and `AQPIM`. Note that `gpu+pq` does not account for PQ compression overhead incurred during prefilling, thus providing an idealistic result.

The elimination of the offloading penalty yields a per-

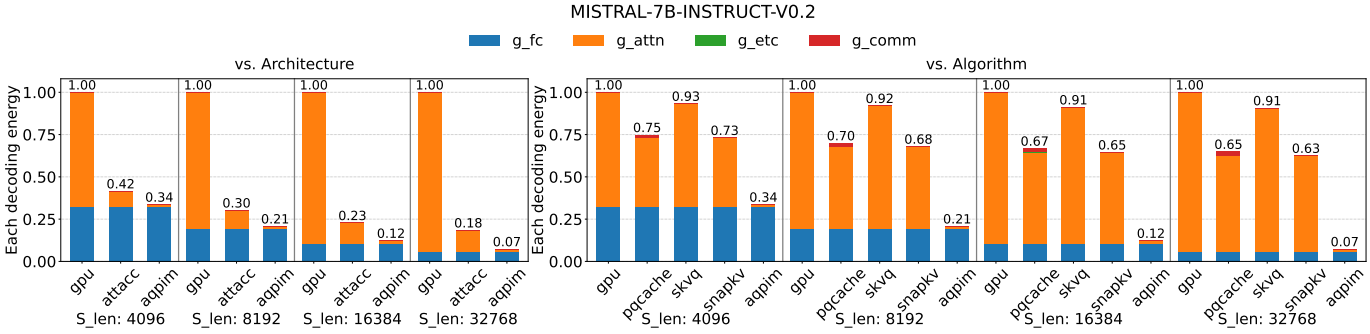


Fig. 14: Normalized energy for decoding comparing different architectures (left) and algorithms (right).

TABLE V: Indirection cycles (Sequenth length = 4K).

	On BankPE	On BufferPE
Key (including transfer to Softmax)	33089	37185
Value	7373	181875

formance gain of 11.39×. This roughly aligns with the gap between GPU's memory bandwidth (3.35TB/s) and PCIe bandwidth (256GB/s). The reduced memory footprint from PQ compression contributes to an additional 5.52× speedup. This is consistent with the high data compressibility of PQ (6.53× reduction of KV capacity). Finally, AQPIM's dedicated architectural optimization yields another 3.85× speedup. While our PIM baseline, AttAcc!, offers 7.2× higher aggregated internal memory bandwidth compared to GPU (3.35TB/s), the un-accelerated operations (e.g. FFN) dominate the decoding latency of AQPIM. However, solely comparing the attention phase observes 10.33× speedup, exceeding the bandwidth gap. This happens because AQPIM can produce more data than a row with a single ACT through data reuse in row buffers during indirection.

When compared to an imaginary AttAcc! (not shown in the figure) that has infinite capacity but the same PE counts, AQPIM achieves 3.4× speedup. While the gain can be explained by the reduced KV size, it is smaller than `gpu+pq`'s 5.52× speedup against `gpu∞` because AttAcc's PIM diminishes the share of attention in the total decoding time.

Lastly, we evaluate the benefits of intra-row indirection. An alternative approach is to perform gather operations on the BufferPE, transferring the entire row data. The results are summarized in Table V. While intra-row indirection at BankPEs significantly reduces off-bank data transfer, performing it on the BufferPE incurs a costly data transfer and resource contention. While the gap is narrower for Keys as they are anyway transferred to BufferPEs for softmax operations, value matrix experiences significant overhead as it necessitates unnecessary roundtrip to the BufferPE between BankPE operations.

F. Accuracy & Speedup vs. Memory Reduction

Figure 15 analyzes the the trade-offs among accuracy, speedup, and memory reduction focusing on the attention kernel. Due to limited space, we show those with the best and worst tradeoffs. In the best-case scenario (left), AQPIM maintains high accuracy even with extreme compression, achieving

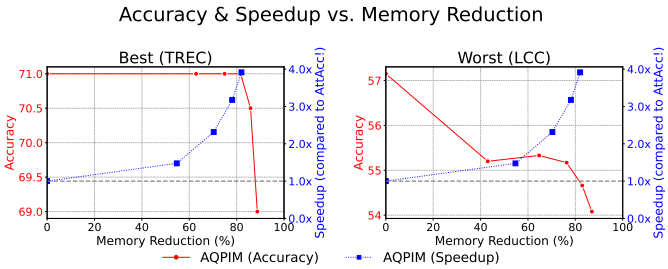


Fig. 15: Accuracy & Speedup vs. Memory Reduction.

a significant speedup. Even in the worst case scenario, AQPIM achieve a comparable accuracy while offering a substantial speedup. Note that compression rate can be flexibly adjusted without hardware modification.

G. Energy Efficiency

We estimated the energy consumption across different approaches. Figure 14 presents the normalized energy consumption per decoding step, varying the input length. Regarding architectural comparisons (left), AQPIM significantly reduces the energy consumed in attention calculation. Compared to the GPU baseline, AQPIM achieves up to a 14.29× improvement in energy efficiency. In terms of algorithmic comparisons (right), AQPIM surpasses other methods in energy efficiency for "attention" components. The energy for "attention" is decreased because AQPIM utilizes a hardware-aware PQ-based attention mechanism, incorporating fixed-size matmul and row buffer retrieval mechanism.

V. Conclusion

In this work, we observe that activations exhibit both locality and similarity, derived from contextual information. Based on the trade-off between compressibility and required memory bandwidth, we propose a clustering-based approach to better exploit the underutilized capabilities of PIM. PQ, a clustering-based method, is well-suited to preserve locality. We further enhance PQ by introducing importance-aware and data-driven optimizations to maintain high accuracy while drastically reducing memory footprint. Additionally, we propose a fast attention computation mechanism that directly operates on the compressed format, enabling localized memory access patterns favorable for PIM.

ACKNOWLEDGEMENTS

This work was supported by JSPS KAKENHI Grant Numbers JP25K03092, JP23H05489, JST BOOST Grant Number JPMJBY24G7, JST PRESTO Grant Number JPMJPR22P7, and JST ALCA-Next Grant Number JPMJAN24F3.

REFERENCES

[1] M. Adnan, A. Arunkumar, G. Jain, P. Nair, I. Soloveychik, and P. Kamath, "Keyformer: Kv cache reduction through key tokens selection for efficient generative inference," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 114–127, 2024.

[2] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, "LongBench: A bilingual, multitask benchmark for long context understanding," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 3119–3137. [Online]. Available: https://aclanthology.org/2024.acl-long.172/

[3] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[4] A. Bulatov, Y. Kuratov, Y. Kapushev, and M. S. Burtsev, "Scaling transformer to 1m tokens and beyond with rmt," 2024. [Online]. Available: https://arxiv.org/abs/2304.11062

[5] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks,(2022)," *arXiv preprint arXiv:2211.12588*, 2022.

[6] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.

[7] P. Chi, S. Li, C. Xu, T. Zhang, J. Zhao, Y. Liu, Y. Wang, and Y. Xie, "Prime: A novel processing-in-memory architecture for neural network computation in reram-based main memory," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 27–39, 2016.

[8] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," *arXiv preprint arXiv:1904.10509*, 2019.

[9] L. T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "Asap7: A 7-nm finfet predictive process design kit," *Microelectronics Journal*, vol. 53, pp. 105–115, 2016. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S002626921630026X

[10] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 16 344–16 359. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/67d57c32e20fd0a7a302cb81d36e40d5-Paper-Conference.pdf

[11] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. ZHANG, and D. Lin, "SKVQ: Sliding-window key and value cache quantization for large language models," in *First Conference on Language Modeling*, 2024. [Online]. Available: https://openreview.net/forum?id=nI6JyFSnyV

[12] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, R. Iyer, D. Sylvester, D. Blaaauw, and R. Das, "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *2018 ACM/IEEE 45Th annual international symposium on computer architecture (ISCA)*. IEEE, 2018, pp. 383–396.

[13] D. Fujiki, "Mvc: Enabling fully coherent multi-data-views through the memory hierarchy with processing in memory," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 800–814.

[14] D. Fujiki, S. Mahlke, and R. Das, "Duality cache for data parallel acceleration," in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 397–410.

[15] D. Fujiki, X. Wang, A. Subramaniyan, and R. Das, *In-/near-memory Computing*. Springer, 2021.

[16] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig, "Pal: Program-aided language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 10 764–10 799.

[17] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-power computer vision*. Chapman and Hall/CRC, 2022, pp. 291–326.

[18] Z. Gou, Z. Shao, Y. Gong, yelong shen, Y. Yang, M. Huang, N. Duan, and W. Chen, "ToRA: A tool-integrated reasoning agent for mathematical problem solving," in *The Twelfth International Conference on Learning Representations*, 2024.

[19] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini, K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh,

K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait, Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[20] S. R. Group, "Ramulator2.0," 2023, https://github.com/CMU-SAFARI/ramulator2.

[21] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[22] M. U. Hadi, R. Qureshi, A. Shah, M. Irfan, A. Zafar, M. B. Shaikh, N. Akhtar, J. Wu, S. Mirjalili *et al.*, "Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects," *Authorea Preprints*, vol. 1, pp. 1–26, 2023.

[23] Y. He, L. Zhang, W. Wu, J. Liu, H. Zhou, and B. Zhuang, "Zipcache: Accurate and efficient KV cache quantization with salient token identification," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=5t4ZAkPiJs

[24] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "Neupims: Npu-pim heterogeneous acceleration for batched llm inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 722–737. [Online]. Available: https://doi.org/10.1145/3620666.3651380

[25] C. Hooper, S. Kim, H. Mohammadzadeh, M. Maheswaran, S. Zhao, J. Paik, M. W. Mahoney, K. Keutzer, and A. Gholami, "Squeezed attention: Accelerating long context length llm inference," 2025. [Online]. Available: https://arxiv.org/abs/2411.09688

[26] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 1270–1303. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/028fcbcf85435d39a40c4d61b42c99a4-Paper-Conference.pdf

[27] W. Hu, H. Zhang, C. Guo, Y. Feng, R. Guan, Z. Hua, Z. Liu, Y. Guan, M. Guo, and J. Leng, "M-ant: Efficient low-bit group quantization for llms via mathematically adaptive numerical type," *arXiv preprint arXiv:2502.18755*, 2025.

[28] L. Huang, S. Cao, N. Parulian, H. Ji, and L. Wang, "Efficient attentions for long document summarization," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 1419–1436. [Online]. Available: https://aclanthology.org/2021.naacl-main.112/

[29] IEA. Energy and AI – analysis. [Online]. Available: https://www.iea.org/reports/energy-and-ai

[30] Intel, "Intel xeon platinum 8480+ processor," 2023, https://www.intel.com/content/www/us/en/products/sku/231746/intel-xeon-platinum-8480-processor-105m-cache-2-00-ghz/specifications.html.

[31] JEDEC, "High bandwidth memory dram (hbm3)," 2022.

[32] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7b," 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[33] H. Jiang, Y. Li, C. Zhang, Q. Wu, X. Luo, S. Ahn, Z. Han, A. Abdi, D. Li, C.-Y. Lin *et al.*, "Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention," *Advances in Neural Information Processing Systems*, vol. 37, pp. 52 481–52 515, 2024.

[34] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011.

[35] J. H. Kim, S.-h. Kang, S. Lee, H. Kim, W. Song, Y. Ro, S. Lee, D. Wang, H. Shin, B. Phuah *et al.*, "Aquabolt-xl: Samsung hbm2-pim with in-memory processing for ml accelerators and beyond," in *2021 IEEE Hot Chips 33 Symposium (HCS)*. IEEE, 2021, pp. 1–26.

[36] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Comput. Archit. Lett.*, vol. 15, no. 1, p. 45–49, Jan. 2016. [Online]. Available: https://doi.org/10.1109/LCA.2015.2414456

[37] T. Kočiský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette, "The NarrativeQA reading comprehension challenge," *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 317–328, 2018. [Online]. Available: https://aclanthology.org/Q18-1023/

[38] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin *et al.*, "Hardware architecture and software stack for pim based on commercial dram technology: Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 43–56.

[39] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware architecture and software stack for pim based on commercial dram technology : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 43–56.

[40] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, "SnapKV: LLM knows what you are looking for before generation," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=poE54GOq2l

[41] H. Lin, H. Xu, Y. Wu, J. Cui, Y. Zhang, L. Mou, L. Song, Z. Sun, and Y. Wei, "Duquant: Distributing outliers via dual transformation makes stronger quantized LLMs," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: https://openreview.net/forum?id=mp8u2Pcmqz

[42] Y. Lin*, H. Tang*, S. Yang*, Z. Zhang, G. Xiao, C. Gan, and S. Han, "Qserve: W4a8kv4 quantization and system co-design for efficient llm serving," *arXiv preprint arXiv:2405.04532*, 2024.

[43] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[44] D. Liu, M. Chen, B. Lu, H. Jiang, Z. Han, Q. Zhang, Q. Chen, C. Zhang, B. Ding, K. Zhang *et al.*, "Retrievalattention: Accelerating long-context llm inference via vector retrieval," *arXiv preprint arXiv:2409.10516*, 2024.

[45] G. Liu, C. Li, J. Zhao, C. Zhang, and M. Guo, "Clusterkv: Manipulating llm kv cache in semantic space for recallable compression," 2024. [Online]. Available: https://arxiv.org/abs/2412.03213

[46] Z.-G. Liu, P. N. Whatmough, Y. Zhu, and M. Mattina, "S2ta: Exploiting structured sparsity for energy-efficient mobile cnn acceleration," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 573–586.

[47] Z. Liu, J. Yuan, H. Jin, S. H. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: a tuning-free asymmetric 2bit quantization for kv cache," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.

[48] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible dram simulator," *IEEE Comput. Archit. Lett.*, vol. 23, no. 1, p. 112–116, Jan. 2024. [Online]. Available: https://doi.org/10.1109/LCA.2023.3333759

[49] L. McInnes, J. Healy, and J. Melville, "Umap: Uniform manifold approximation and projection for dimension reduction," 2020. [Online]. Available: https://arxiv.org/abs/1802.03426

[50] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=Byj72udxe

[51] NVIDIA, "Nvidia h100 tensor core gpu," 2024, https://arc.net/l/quote/btwhvenw.

[52] OpenAI, "Models," 2024, https://platform.openai.com/docs/models/gpt-4o.

[53] M. Oren, M. Hassid, N. Yarden, Y. Adi, and R. Schwartz, "Transformers are multi-state rnns," *arXiv preprint arXiv:2401.06104*, 2024.

[54] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," *ACM SIGARCH computer architecture news*, vol. 45, no. 2, pp. 27–40, 2017.

[55] J. Park and J. Choi, "attacc_simulator," 2024, https://github.com/scale-snu/attacc_simulator.

[56] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "Attacc! unleashing the power of pim for batched transformer-based generative model inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 103–119. [Online]. Available: https://doi.org/10.1145/3620665.3640422

[57] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 14–26, 2016.

[58] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "Omniquant: Omnidirectionally calibrated quantization for large language models," in *ICLR*, 2024. [Online]. Available: https://openreview.net/forum?id=8Wuvhh0LYW

[59] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Re, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proceedings of the 40th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, and J. Scarlett, Eds., vol. 202. PMLR, 23–29 Jul 2023, pp. 31 094–31 116. [Online]. Available: https://proceedings.mlr.press/v202/sheng23a.html

[60] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, *Efficient processing of deep neural networks*. Springer, 2020.

[61] G. Team, P. Georgiev, V. I. Lei, R. Burnell, L. Bai, A. Gulati, G. Tanzer, D. Vincent, Z. Pan, S. Wang, S. Mariooryad, Y. Ding, X. Geng, F. Alcober, R. Frostig, M. Omernick, L. Walker, C. Paduraru, C. Sorokin, A. Tacchetti, C. Gaffney, S. Daruki, O. Sercinoglu, Z. Gleicher, J. Love, P. Voigtlaender, R. Jain, G. Surita, K. Mohamed, R. Blevins, J. Ahn, T. Zhu, K. Kawintiranon, O. Firat, Y. Gu, Y. Zhang, M. Rahtz, M. Faruqui, N. Clay, J. Gilmer, J. Co-Reyes, I. Penchev, R. Zhu, N. Morioka, K. Hui, K. Haridasan, V. Campos, M. Mahdieh, M. Guo, S. Hassan, K. Kilgour, A. Vezer, H.-T. Cheng, R. de Liedekerke, S. Goyal, P. Barham, D. Strouse, S. Noury, J. Adler, M. Sundararajan, S. Vikram, D. Lepikhin, M. Paganini, X. Garcia, F. Yang, D. Valter, M. Trebacz, K. Vodrahalli, C. Asawaroengchai, R. Ring, N. Kalb, L. B. Soares, S. Brahma, D. Steiner, T. Yu, F. Mentzer, A. He, L. Gonzalez, B. Xu, R. L. Kaufman, L. E. Shafey, J. Oh, T. Hennigan, G. van den Driessche, S. Odoom, M. Lucic, B. Roelofs, S. Lall, A. Marathe, B. Chan, S. Ontanon, L. He, D. Teplyashin, J. Lai, P. Crone, B. Damoc, L. Ho, S. Riedel, K. Lenc, C.-K. Yeh, A. Chowdhery, Y. Xu, M. Kazemi, E. Amid, A. Petrushkina, K. Swersky, A. Khodaei, G. Chen, C. Larkin, M. Pinto, G. Yan, A. P. Badia, P. Patil, S. Hansen, D. Orr, S. M. R. Arnold, J. Grimstad, A. Dai, S. Douglas, R. Sinha, V. Yadav, X. Chen, E. Gribovskaya, J. Austin, J. Zhao, K. Patel, P. Komarek, S. Austin, S. Borgeaud, L. Friso, A. Goyal, B. Caine, K. Cao, D.-W. Chung, M. Lamm, G. Barth-Maron, T. Kagohara, K. Olszewska, M. Chen, K. Shivakumar, R. Agarwal, H. Godhia, R. Rajwar, J. Snaider, X. Dotiwalla, Y. Liu, A. Barua, V. Ungureanu, Y. Zhang, B.-O. Batsaikhan, M. Wirth, J. Qin, I. Danihelka, T. Doshi, M. Chadwick, J. Chen, S. Jain, Q. Le, A. Kar, M. Gurumurthy, C. Li, R. Sang, F. Liu, L. Lamprou, R. Munoz, N. Lintz, H. Mehta, H. Howard, M. Reynolds, L. Aroyo, Q. Wang, L. Blanco, A. Cassirer, J. Griffith, D. Das, S. Lee, J. Sygnowski, Z. Fisher, J. Besley, R. Powell, Z. Ahmed, D. Paulus, D. Reitter, Z. Borsos, R. Joshi, A. Pope, S. Hand, V. Selo, V. Jain, N. Sethi, M. Goel, T. Makino, R. May, Z. Yang, J. Schalkwyk, C. Butterfield, A. Hauth, A. Goldin, W. Hawkins, E. Senter, S. Brin, O. Woodman, M. Ritter, E. Noland, M. Giang, V. Bolina, L. Lee, T. Blyth, I. Mackinnon, M. Reid, O. Sarvana, D. Silver, A. Chen, L. Wang, L. Maggiore, O. Chang, N. Attaluri, G. Thornton, C.-C. Chiu, O. Bunyan, N. Levine, T. Chung, E. Eltyshev, X. Si, T. Lillicrap, D. Brady, V. Aggarwal, B. Wu, Y. Xu, R. McIlroy, K. Badola, P. Sandhu, E. Moreira, W. Stokowiec, R. Hemsley, D. Li, A. Tudor, P. Shyam, E. Rahimtoroghi, S. Haykal, P. Sprechmann, X. Zhou, D. Mincu, Y. Li, R. Addanki, K. Krishna, X. Wu, A. Frechette, M. Eyal, A. Dafoe, D. Lacey, J. Whang, T. Avrahami, Y. Zhang, E. Taropa, H. Lin, D. Toyama, E. Rutherford, M. Sano, H. Choe, A. Tomala, C. Safranek-Shrader, N. Kassner, M. Pajarskas, M. Harvey, S. Sechrist, M. Fortunato, C. Lyu, G. Elsayed, C. Kuang, J. Lottes, E. Chu, C. Jia, C.-W. Chen, P. Humphreys, K. Baumli, C. Tao, R. Samuel, C. N. dos Santos, A. Andreassen, N. Rakićević, D. Grewe, A. Kumar, S. Winkler, J. Caton, A. Brock, S. Dalmia, H. Sheahan, I. Barr, Y. Miao, P. Natsev, J. Devlin, F. Behbahani, F. Prost, Y. Sun, A. Myaskovsky, T. S. Pillai, D. Hurt, A. Lazaridou, X. Xiong, C. Zheng, F. Pardo, X. Li, D. Horgan, J. Stanton, M. Ambar, F. Xia, A. Lince, M. Wang, B. Mustafa, A. Webson, H. Lee, R. Anil, M. Wicke, T. Dozat, A. Sinha, E. Piqueras, E. Dabir, S. Upadhyay, A. Boral, L. A. Hendricks, C. Fry, J. Djolonga, Y. Su, J. Walker, J. Labanowski, R. Huang, V. Misra, J. Chen, R. Skerry-Ryan, A. Singh, S. Rijhwani, D. Yu, A. Castro-Ros, B. Changpinyo, R. Datta, S. Bagri, A. M. Hrafnkelsson, M. Maggioni, D. Zheng, Y. Sulsky, S. Hou, T. L. Paine, A. Yang, J. Riesa, D. Rogozinska, D. Marcus, D. E. Badawy, Q. Zhang, L. Wang, H. Miller, J. Greer, L. L. Sjos, A. Nova, H. Zen, R. Chaabouni, M. Rosca, J. Jiang, C. Chen, R. Liu, T. Sainath, M. Krikun, A. Polozov, J.-B. Lespiau, J. Newlan, Z. Cankara, S. Kwak, Y. Xu, P. Chen, A. Coenen, C. Meyer, K. Tsihlas, A. Ma, J. Gottweis, J. Xing, C. Gu, J. Miao, C. Frank, Z. Cankara, S. Ganapathy, I. Dasgupta, S. Hughes-Fitt, H. Chen, D. Reid, K. Rong, H. Fan, J. van Amersfoort, V. Zhuang, A. Cohen, S. S. Gu, A. Mohananey, A. Ilic, T. Tobin, J. Wieting, A. Bortsova, P. Thacker, E. Wang, E. Caveness, J. Chiu, E. Sezener, A. Kaskasoli, S. Baker, K. Millican, M. Elhawaty, K. Aisopos, C. Lebsack, N. Byrd, H. Dai, W. Jia, M. Wiethoff, E. Davoodi, A. Weston, L. Yagati, A. Ahuja, I. Gao, G. Pundak, S. Zhang, M. Azzam, K. C. Sim, S. Caelles, J. Keeling, A. Sharma, A. Swing, Y. Li, C. Liu, C. G. Bostock, Y. Bansal, Z. Nado, A. Anand, J. Lipschultz, A. Karmarkar, L. Proleev, A. Ittycheriah, S. H. Yeganeh, G. Polovets, A. Faust, J. Sun, A. Rrustemi, P. Li, R. Shivanna, J. Liu, C. Welty, F. Lebron, A. Baddepudi, S. Krause, E. Parisotto, R. Soricut, Z. Xu, D. Bloxwich, M. Johnson, B. Neyshabur, J. Mao-Jones, R. Wang, V. Ramasesh, Z. Abbas, A. Guez, C. Segal, D. D. Nguyen, J. Svensson, L. Hou, S. York, K. Milan, S. Bridgers, W. Gworek, M. Tagliasacchi, J. Lee-Thorp, M. Chang, A. Guseynov, A. J. Hartman, M. Kwong, R. Zhao, S. Kashem, E. Cole, A. Miech, R. Tanburn, M. Phuong, F. Pavetic, S. Cevey, R. Comanescu, R. Ives, S. Yang, C. Du, B. Li, Z. Zhang, M. Iinuma, C. H. Hu, A. Roy, S. Bijwadia, Z. Zhu, D. Martins, R. Saputro, A. Gergely, S. Zheng, D. Jia, I. Antonoglou, A. Sadovsky, S. Gu, Y. Bi, A. Andreev, S. Samangooei, M. Khan, T. Kocisky, A. Filos, C. Kumar, C. Bishop, A. Yu, S. Hodkinson, S. Mittal, P. Shah, A. Moufarek, Y. Cheng, A. Bloniarz, J. Lee, P. Pejman, P. Michel, S. Spencer, V. Feinberg, X. Xiong, N. Savinov, C. Smith, S. Shakeri, D. Tran, M. Chesus, B. Bohnet, G. Tucker, T. von Glehn, C. Muir, Y. Mao, H. Kazawa, A. Slone, K. Soparkar, D. Shrivastava, J. Cobon-Kerr, M. Sharman, J. Pavagadhi, C. Araya, K. Misiunas, N. Ghelani, M. Laskin, D. Barker, Q. Li, A. Briukhov, N. Houlsby, M. Glaese, B. Lakshminarayanan, N. Schucher, Y. Tang, E. Collins, H. Lim, F. Feng, A. Recasens, G. Lai, A. Magni, N. D. Cao, A. Siddhant, Z. Ashwood, J. Orbay, M. Dehghani, J. Brennan, Y. He, K. Xu, Y. Gao, C. Saroufim, J. Molloy, X. Wu, S. Arnold, S. Chang, J. Schrittwieser, E. Buchatskaya, S. Radpour, M. Polacek, S. Giordano, A. Bapna, S. Tokumine, V. Hellendoorn, T. Sottiaux, S. Cogan, A. Severyn, M. Saleh, S. Thakoor, L. Shefey, S. Qiao, M. Gaba, S. yiin Chang, C. Swanson, B. Zhang, B. Lee, P. K. Rubenstein,

G. Song, T. Kwiatkowski, A. Koop, A. Kannan, D. Kao, P. Schuh, A. Stjerngren, G. Ghiasi, G. Gibson, L. Vilnis, Y. Yuan, F. T. Ferreira, A. Kamath, T. Klimenko, K. Franko, K. Xiao, I. Bhattacharya, M. Patel, R. Wang, A. Morris, R. Strudel, V. Sharma, P. Choy, S. H. Hashemi, J. Landon, M. Finkelstein, P. Jhakra, J. Frye, M. Barnes, M. Mauger, D. Daun, K. Baatarsukh, M. Tung, W. Farhan, H. Michalewski, F. Viola, F. de Chaumont Quitry, C. L. Lan, T. Hudson, Q. Wang, F. Fischer, I. Zheng, E. White, A. Dragan, J. baptiste Alayrac, E. Ni, A. Pritzel, A. Iwanicki, M. Isard, A. Bulanova, L. Zilka, E. Dyer, D. Sachan, S. Srinivasan, H. Muckenhirn, H. Cai, A. Mandhane, M. Tariq, J. W. Rae, G. Wang, K. Ayoub, N. FitzGerald, Y. Zhao, W. Han, C. Alberti, D. Garrette, K. Krishnakumar, M. Gimenez, A. Levskaya, D. Sohn, J. Matak, I. Iturrate, M. B. Chang, J. Xiang, Y. Cao, N. Ranka, G. Brown, A. Hutter, V. Mirrokni, N. Chen, K. Yao, Z. Egyed, F. Galilee, T. Liechty, P. Kallakuri, E. Palmer, S. Ghemawat, J. Liu, D. Tao, C. Thornton, T. Green, M. Jasarevic, S. Lin, V. Cotruta, Y.-X. Tan, N. Fiedel, H. Yu, E. Chi, A. Neitz, J. Heitkaemper, A. Sinha, D. Zhou, Y. Sun, C. Kaed, B. Hulse, S. Mishra, M. Georgaki, S. Kudugunta, C. Farabet, I. Shafran, D. Vlasic, A. Tsitsulin, R. Ananthanarayanan, A. Carin, G. Su, P. Sun, S. V, G. Carvajal, J. Broder, I. Comsa, A. Repina, W. Wong, W. W. Chen, P. Hawkins, E. Filonov, L. Loher, C. Hirnschall, W. Wang, J. Ye, A. Burns, H. Cate, D. G. Wright, F. Piccinini, L. Zhang, C.-C. Lin, I. Gog, Y. Kulizhskaya, A. Sreevatsa, S. Song, L. C. Cobo, A. Iyer, C. Tekur, G. Garrido, Z. Xiao, R. Kemp, H. S. Zheng, H. Li, A. Agarwal, C. Ngani, K. Goshvadi, R. Santamaria-Fernandez, W. Fica, X. Chen, C. Gorgolewski, S. Sun, R. Garg, X. Ye, S. M. A. Eslami, N. Hua, J. Simon, P. Joshi, Y. Kim, I. Tenney, S. Potluri, L. N. Thiet, Q. Yuan, F. Luisier, A. Chronopoulou, S. Scellato, P. Srinivasan, M. Chen, V. Koverkathu, V. Dalibard, Y. Xu, B. Saeta, K. Anderson, T. Sellam, N. Fernando, F. Huot, J. Jung, M. Varadarajan, M. Quinn, A. Raul, M. Le, R. Habalov, J. Clark, K. Jalan, K. Bullard, A. Singhal, T. Luong, B. Wang, S. Rajayogam, J. Eisenschlos, J. Jia, D. Finchelstein, A. Yakubovich, D. Balle, M. Fink, S. Agarwal, J. Li, D. Dvijotham, S. Pal, K. Kang, J. Konzelmann, J. Beattie, O. Dousse, D. Wu, R. Crocker, C. Elkind, S. R. Jonnalagadda, J. Lee, D. Holtmann-Rice, K. Kallarackal, R. Liu, D. Vnukov, N. Vats, L. Invernizzi, M. Jafari, H. Zhou, L. Taylor, J. Prendki, M. Wu, T. Eccles, T. Liu, K. Kopparapu, F. Beaufays, C. Angermueller, A. Marzoca, S. Sarcar, H. Dib, J. Stanway, F. Perbet, N. Trdin, R. Sterneck, A. Khorlin, D. Li, X. Wu, S. Goenka, D. Madras, S. Goldshtein, W. Gierke, T. Zhou, Y. Liu, Y. Liang, A. White, Y. Li, S. Singh, S. Bahargam, M. Epstein, S. Basu, L. Lao, A. Ozturel, C. Crous, A. Zhai, H. Lu, Z. Tung, N. Gaur, A. Walton, L. Dixon, M. Zhang, A. Globerson, G. Uy, A. Bolt, O. Wiles, M. Nasr, I. Shumailov, M. Selvi, F. Piccinno, R. Aguilar, S. McCarthy, M. Khalman, M. Shukla, V. Galic, J. Carpenter, K. Villela, H. Zhang, H. Richardson, J. Martens, M. Bosnjak, S. R. Belle, J. Seibert, M. Alnahlawi, B. McWilliams, S. Singh, A. Louis, W. Ding, D. Popovici, L. Simicich, L. Knight, P. Mehta, N. Gupta, C. Shi, S. Fatehi, J. Mitrovic, A. Grills, J. Pagadora, T. Munkhdalai, D. Petrova, D. Eisenbud, Z. Zhang, D. Yates, B. Mittal, N. Tripuraneni, Y. Assael, T. Brovelli, P. Jain, M. Velimirovic, C. Akbulut, J. Mu, W. Macherey, R. Kumar, J. Xu, H. Qureshi, G. Comanici, J. Wiesner, Z. Gong, A. Ruddock, M. Bauer, N. Felt, A. GP, A. Arnab, D. Zelle, J. Rothfuss, B. Rosgen, A. Shenoy, B. Seybold, X. Li, J. Mudigonda, G. Erdogan, J. Xia, J. Simsa, A. Michi, Y. Yao, C. Yew, S. Kan, I. Caswell, C. Radebaugh, A. Elisseeff, P. Valenzuela, K. McKinney, K. Paterson, A. Cui, E. Latorre-Chimoto, S. Kim, W. Zeng, K. Durden, P. Ponnapalli, T. Sosea, C. A. Choquette-Choo, J. Manyika, B. Robenek, H. Vashisht, S. Pereira, H. Lam, M. Velic, D. Owusu-Afriyie, K. Lee, T. Bolukbasi, A. Parrish, S. Lu, J. Park, B. Venkatraman, A. Talbert, L. Rosique, Y. Cheng, A. Sozanschi, A. Paszke, P. Kumar, J. Austin, L. Li, K. Salama, B. Perz, W. Kim, N. Dukkipati, A. Baryshnikov, C. Kaplanis, X. Sheng, Y. Chervonyi, C. Unlu, D. de Las Casas, H. Askham, K. Tunyasuvunakool, F. Gimeno, S. Poder, C. Kwak, M. Miecnikowski, V. Mirrokni, A. Dimitriev, A. Parisi, D. Liu, T. Tsai, T. Shevlane, C. Kouridi, D. Garmon, A. Goedeckemeyer, A. R. Brown, A. Vijayakumar, A. Elqursh, S. Jazayeri, J. Huang, S. M. Carthy, J. Hoover, L. Kim, S. Kumar, W. Chen, C. Biles, G. Bingham, E. Rosen, L. Wang, Q. Tan, D. Engel, F. Pongetti, D. de Cesare, D. Hwang, L. Yu, J. Pullman, S. Narayanan, K. Levin, S. Gopal, M. Li, A. Aharoni, T. Trinh, J. Lo, N. Casagrande, R. Vij, L. Matthey, B. Ramadhana, A. Matthews, C. Carey, M. Johnson, K. Goranova, R. Shah, S. Ashraf, K. Dasgupta, R. Larsen, Y. Wang, M. R. Vuyyuru, C. Jiang, J. Ijazi, K. Osawa, C. Smith, R. S. Boppana, T. Bilal, Y. Koizumi, Y. Xu, Y. Altun, N. Shabat, B. Bariach, A. Korchemniy, K. Choo, O. Ronneberger, C. Iwuanyanwu, S. Zhao, D. Soergel, C.-J. Hsieh, I. Cai, S. Iqbal, M. Sundermeyer, Z. Chen, E. Bursztein, C. Malaviya, F. Biadsy, P. Shroff, I. Dhillon, T. Latkar, C. Dyer, H. Forbes, M. Nicosia, V. Nikolaev, S. Greene, M. Georgiev, P. Wang, N. Martin, H. Sedghi, J. Zhang, P. Banzal, D. Fritz, V. Rao, X. Wang, J. Zhang, V. Patraucean, D. Du, I. Mordatch, I. Jurin, L. Liu, A. Dubey, A. Mohan, J. Nowakowski, V.-D. Ion, N. Wei, R. Tojo, M. A. Raad, D. A. Hudson, V. Keshava, S. Agrawal, K. Ramirez, Z. Wu, H. Nguyen, J. Liu, M. Sewak, B. Petrini, D. Choi, I. Philips, Z. Wang, I. Bica, A. Garg, J. Wilkiewicz, P. Agrawal, X. Li, D. Guo, E. Xue, N. Shaik, A. Leach, S. M. Khan, J. Wiesinger, S. Jerome, A. Chakladar, A. W. Wang, T. Ornduff, F. Abu, A. Ghaffarkhah, M. Wainwright, M. Cortes, F. Liu, J. Maynez, A. Terzis, P. Samangouei, R. Mansour, T. Kepa, F.-X. Aubet, A. Algymr, D. Banica, A. Weisz, A. Orban, A. Senges, E. Andrejczuk, M. Geller, N. D. Santo, V. Anklin, M. A. Merey, M. Baeuml, T. Strohman, J. Bai, S. Petrov, Y. Wu, D. Hassabis, K. Kavukcuoglu, J. Dean, and O. Vinyals, "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context," 2024. [Online]. Available: https://arxiv.org/abs/2403.05530

[62] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.

[63] Y. Wu, M. N. Rabe, D. Hutchins, and C. Szegedy, "Memorizing transformers," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=TrjbxzRcnf-

[64] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: accurate and efficient post-training quantization for large language models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.

[65] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient streaming language models with attention sinks," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=NG7sS51zVF

[66] D. Xu, I. W. Tsang, and Y. Zhang, "Online product quantization," *IEEE Transactions on Knowledge and Data Engineering*, vol. 30, no. 11, pp. 2185–2198, 2018.

[67] J. Yuan, H. Gao, D. Dai, J. Luo, L. Zhao, Z. Zhang, Z. Xie, Y. X. Wei, L. Wang, Z. Xiao, Y. Wang, C. Ruan, M. Zhang, W. Liang, and W. Zeng, "Native sparse attention: Hardware-aligned and natively trainable sparse attention," 2025. [Online]. Available: https://arxiv.org/abs/2502.11089

[68] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "Gobo: Quantizing attention-based nlp models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.

[69] A. H. Zadeh, M. Mahmoud, A. Abdelhadi, and A. Moshovos, "Mokey: enabling narrow fixed-point inference for out-of-the-box floating-point transformer models," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ser. ISCA '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 888–901. [Online]. Available: https://doi.org/10.1145/3470496.3527438

[70] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, "Big bird: Transformers for longer sequences," *Advances in neural information processing systems*, vol. 33, pp. 17 283–17 297, 2020.

[71] Zenodo, https://zenodo.org/records/17378113.

[72] H. Zhang, X. Ji, Y. Chen, F. Fu, X. Miao, X. Nie, W. Chen, and B. Cui, "Pqcache: Product quantization-based kvcache for long context llm inference," 2024. [Online]. Available: https://arxiv.org/abs/2407.12820

[73] T. Zhang, J. Yi, Z. Xu, and A. Shrivastava, "Kv cache is 1 bit per channel: Efficient large language model inference with coupled quantization," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 3304–3331. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/05d6b5b6901fb57d2c287e1d3ce6d63c-Paper-Conference.pdf

[74] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Re, C. Barrett, Z. Wang, and B. Chen, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," in *Thirty-seventh Conference on Neural*

Information Processing Systems, 2023. [Online]. Available: https://openreview.net/forum?id=RkRrPp7GKO

[75] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasikci, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.

[76] Y. Zhao, D. Wu, and J. Wang, "Alisa: Accelerating large language model inference via sparsity-aware kv caching," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1005–1017.

[77] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 1556–1577, 2024.